

Design Rationale Req1

Design 1

1. Each plant species (e.g., Wild Apple Tree, Yew Berry Tree) and their growth stages were created and linked manually inside the Earth class.
2. For example, in the forest map, I initialize AppleSprouts and linked the next stage to AppleSapling and then to MatureAppleTree (Wild Apple Tree).
3. These directly linking techniques were applied on both forest and plains maps.
4. Based on the situation (e.g., on Plains map it will skip the sapling stage).

Pro	Cons
Straightforward and explicit	Will lead to connascence of execution because the order of creating tree and linking must be correct if not the setup fails and will crash the program.
Each stage creation is transparent and easy to trace	Violate DRY principle, similar logic repeated multiple times (e.g., setup tree type)
Minimal abstraction, the flow is easy to understand	Not easy for future extensibility (e.g., adding new tree species or modifying growth rules need editing several sections of code). Moreover, hard to maintain consistency if additional stages are introduced later.
	Violate SRP (e.g., creation and logic are mixed)

Design 2

1. Introduce two interface act as factory and corresponding child classes:
 - a. CreateAppleTrees
 - b. CreateYewBerryTrees
 - c. AppleChild
 - d. YewBerryChild
2. Each responsible for building and manage its own species growth lifecycle.
3. AppleChild and YewBerryChild now can then inherit Growthable abstract class and letting AppleSapling, AppleSprouts, and YewBerrySapling inherit them respectively.
4. Growthable which then handle the common logic such as time based growth, random transformation chance, and fruit production logic (with ProduceableFruitTree).

5. Each factory method automatically create all possible growth stage, by setting their nextStage, and return the initial stage.
6. The earth class then simply inject the return tree onto the map ground.

Pro	Cons
Eliminate connascence of execution	Required more understanding of the factory pattern
Follow Object-Oriented-Principle	Each new tree species must define its own factory interface and child class.
Ensuring follow SOLID principle: 1. SRP→ Each class(e.g., AppleChild, YewBerryChild) focus on its tree type only. 2. OCP→ New tree species can be added by introducing new factory interface and child classes without modify existing ones. 3. LSP→All subclass of Growthable behave consistently under the same interface 4. ISP and DIP → Classes depend on abstraction (CreateAppleTrees, CreateYewBerryTrees, SummonApple, SummonYewBerry) instead of concrete implementations. It uses static factory methods to provide creation functionality without requiring inheritance or instance level behaviour.	
Avoid violating DRY, by removing repetitive linking logic by using setup in factory method.	
Follow KISS, the naming variable, code logic, etc..., ensure simple and readable avoiding unnecessary abstraction layer while solving the growth linking issues in a more effective way.	
Low coupling and high cohesion, factory only handle creation and each class manages one clear responsibility.	

Chosen Design:

Design 2,

Because it has more clarity, maintainability, and scalability. Helps to eliminate condescension of execution by internalizing the creation sequence within interface factory, it also helps to error-free the initialization.

It also satisfies the OOP, SOLID, DRY, KISS principles. Help to providing extensible code that can easily accommodate future plant species or new growth rule without breaking existing functionality.

Existing classes and responsibilities:

1. Interface:

- a. CreateAppleTrees → Factory interface to create and chain apple tree stages.
- b. CreateYewBerryTrees → Same for YewBerry trees.
- c. SummonApple/SummonYewBerry → Injector to return new instance of fruit.

2. Abstract Classes:

- a. ProduceableFruitTree → Handle drop fruit timing on its surrounding.
- b. Growthable → Manage time, transformation rate and growth logic and it able to produce fruit or not.
- c. AppleChild, YewBerryChild → Manages apple type/yew berry tree type growth and fruit behavior; it used CreateAppleTrees/CreateYewBerryTrees and SummonApple/SummonYewBerry respectively (static method interface factory).

3. Concrete classes

- a. AppleSprouts, AppleSapling, AppleTree → Stages of Apple tree growth
- b. YewBerrySapling, YewBerryTree → Stages of YewBerry growth.
- c. Earth → Inject the correct tree type into maps (Forest/Plains).

Classes Relate to and Interact with the Existing System

1. Earth injected tree via static factory calls (AppleChild, YewBerryChild) into the correct map with correct behaviour.
2. Growthable.tick() manages transformation and fruit production with each turn.
3. setGround(nextStage) then replace the current ground type to its next stages (automatically growth).
4. ProduceableFruitTree will then spawn the fruit into its surroundings through static interface injector (SummonApple, SummonYewBerry).

Deliver Functionality

1. Earth plant trees based on map type
2. Growthable instance track growth time

3. Each tick, transformation is check and if able then transform
4. Successful growth then replaces the ground to next stage.
5. Fruit drop if can produce is true
6. YewBerry trees will check 50% every three turn if success then only transform.

Conclusions

Second design removed connascence of execution by encapsulating setup logic within factory interfaces implemented by each species child class. It adheres with OOP, SOLID, DRY, KISS. Which makes it easier for future extension. By converting SummonApple and SummonYewBerry into static factory style interfaces instead of default, further simplifies dependency management and adheres to KISS principle and promote low coupling (simply call the static method).